Getting Started with Colab (and Python)

CS 540 - Principles of AI - Catholic Polytechnic U, summer 2026

Table of Contents

Objectives

Let's make a simple plot with Python in Colab and learn a few things about Python along the way.

1. Import `pandas` library for data frame manipulation.
2. Download the `Iris` dataset from Kaggle.
3. Load dataset into a `pandas` `DataFrame`.
4. Include exception handling with `try` `except`.
5. Explore the dataset
6. Create a simple plot
7. Summarize and check your understanding

Along the way, you're also going to pick up:

- Create text cells with markdown language.
- Create and run code cells.
- Use the terminal (aka shell).
- Understand Colab controls and settings.
- Use Gemini AI in Colab.

You can find the complete demo in Colab here: tinyurl.com/colab-python-solution

Colab controls

- The notebook consists of cells in markdown (text) or code (Python).
- The markdown cells produce text with minimal layout
- The code cells produce output (text or graphics)

A few available keys:

Shortcut	Action
`Enter`	edit the selected cell
`Esc`	leave edit mode
`Ctrl+M Y`	change cell to Code
`Ctrl+M M`	change cell to Markdown
`Shift+Enter`	run cell, go to next
`Ctrl+Enter`	run cell, stay
`Ctrl+M B`	insert cell below
`Ctrl+M D`	delete cell
`Ctrl+M Z`	undo cell operation
`Ctrl+S`	save notebook

Other things you can do:

1. view a table of contents
2. Find and replace
3. Filter for code snippets
4. Inspect/explore datasets
5. View, import, export, delete files

Import `pandas` library

- Import `pandas` library for data frame manipulation.
- If `pandas` is not available, you must first install it using the `pip` Python package manager:

```
pip install pandas
```

```
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: pandas in /home/aletheia/.local/lib/python3.10/site
Requirement already satisfied: python-dateutil>=2.8.2 in /home/aletheia/.local/lib
Requirement already satisfied: numpy>=1.22.4 in /home/aletheia/.local/lib/python3.
Requirement already satisfied: tzdata>=2022.7 in /home/aletheia/.local/lib/python3
Requirement already satisfied: pytz>=2020.1 in /usr/lib/python3/dist-packages (fro
Requirement already satisfied: six>=1.5 in /usr/lib/python3/dist-packages (from py
```

- In Colab, you can run shell (`bash(1)`) commands in iPython code cells by putting a `!` in front of the command:

```
!pip install pandas
```

- Alternatively, you can also run this command (without the `!`) directly on the shell by opening the `Terminal` at the bottom:

```
which pip
pip install pandas
exit
```

```
/usr/bin/pip
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: pandas in /home/aletheia/.local/lib/python3.10/site
Requirement already satisfied: tzdata>=2022.7 in /home/aletheia/.local/lib/python3
Requirement already satisfied: pytz>=2020.1 in /usr/lib/python3/dist-packages (fro
Requirement already satisfied: numpy>=1.22.4 in /home/aletheia/.local/lib/python3.
Requirement already satisfied: python-dateutil>=2.8.2 in /home/aletheia/.local/lib
Requirement already satisfied: six>=1.5 in /usr/lib/python3/dist-packages (from py
```

Optional: pip provenance (ask Gemini) `skip`

- But where does `pip` get the package from? Open **Gemini** and ask it:

  > Where does `pip` get the Python package from?

- Answer (likely slightly different in your case):

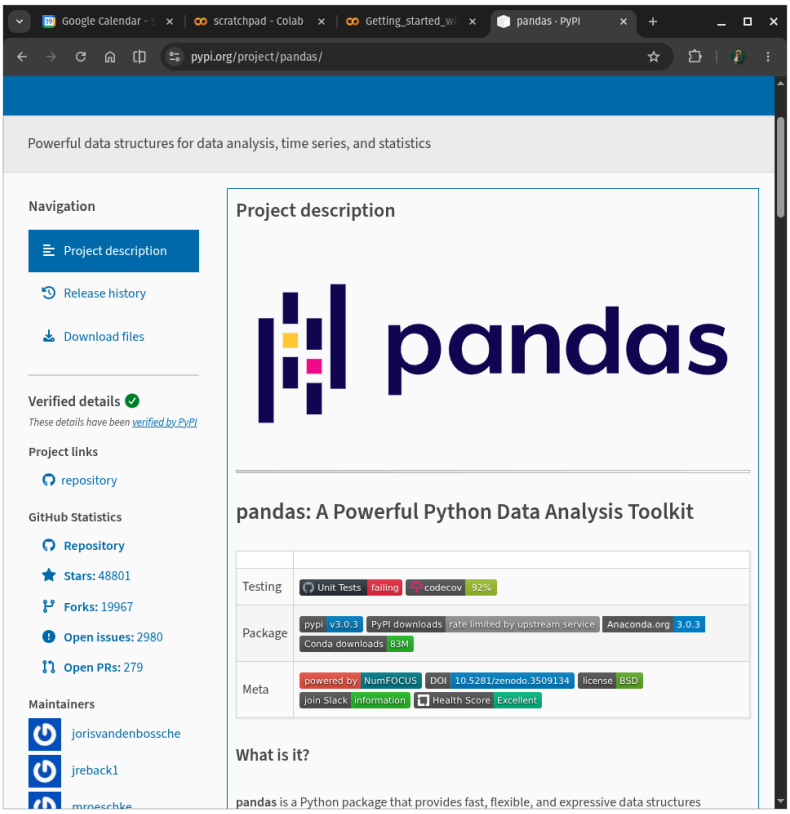
  > When you use `pip install pandas`, `pip` typically retrieves the pandas package from the Python Package Index (PyPI), which is the official third-party software repository for Python. It then installs the package and its dependencies into your Python environment.

- The AI will answer no-code questions in the console. This is a useful addition to the notebook, so continue:

  > Can you put that answer straight into my notebook in a text cell, please.

- The explanation should now appear at the end of your notebook. To keep it there, you have to `Accept` it. You may have to move the block around to be in the right place.

- Create another browser tab and open `pypi.org`, then `search` for `pandas`, and you get to this page: https://pypi.org/project/pandas/



Import pandas

- We can now `import` the `pandas` library into the current Python session.

```
import pandas as pd
```

Optional: session internals - `sys.modules`, `dir()` `skip`

- There is an existing `iPython` session running in the background. When it was started, a namespace was created, which is populated with user-defined objects like loaded libraries or variables[1].
- To check that the library is loaded and available in the current session:

```
import sys
print('pandas' in sys.modules)
```

```
True
```

- In this command, Python looks for the string `pandas` in the `keys` of the dictionary. If it is found (because it was loaded), then a Boolean value, `True`, is returned.
- Detailed explanation:

`sys.modules` is a dictionary that tracks all modules ever loaded during the current Python session. It maps its **keys** - module names, e.g. `'pandas'` (`str`) - to its **values**, the module objects - e.g. `<module 'pandas' from '...'>`

When you write `'pandas' in sys.modules`:

1. Python sees `sys.modules` is a dict-like object.
2. It calls `sys.modules.__contains__('pandas')`.
3. Since `sys.modules` is a `dict`, `__contains__` checks the keys for `'pandas'`.
4. Returns `True` if pandas was imported earlier in the session, `False` otherwise.

- You can also check what names `pandas` put into your namespace (analog to `data()` in R):

```
print(dir(pd))
```

```
['ArrowDtype', 'BooleanDtype', 'Categorical', 'CategoricalDtype', 'CategoricalInde
```

- This is hard to check. You can filter over the printed data with a list comprehension:

```
print(sorted([m for m in sys.modules if 'pandas' in m]))
```

```
['pandas', 'pandas._config', 'pandas._config.config', 'pandas._config.dates', 'pan
```

This prints a sorted list of all module names in `sys.modules` that contain the substring `'pandas'`. We're going to to some I/O but that submodule, pandas.io, is only loaded once we perform I/O, e.g. when we use `pd.read_csv` to read CSV data into a data frame.

- The list comprehension is a one line version of a loop:

```
import sys # import system function library
import pandas as pd # import pandas function library (DataFrames)
for m in sorted(sys.modules): # loop over the sorted list of loaded
                              # system modules
    if 'pandas' in m: # check if module name contains the string
                      # 'pandas'
        print(m, end = ', ')
```

```
pandas, pandas._config, pandas._config.config, pandas._config.dates, pandas._confi
```

Download CSV data from `kaggle.com` into a `DataFrame`

- Download the `Iris` dataset from Kaggle.

The `Iris` dataset is a classic for introductory machine learning and data science. You can download it directly from a URL - I have saved the long address in a tinyurl:

```
data_url = 'https://tinyurl.com/iris-data-csv' # raw csv data
```

- Load dataset into a `pandas` `DataFrame`.

  The quickest way to load the CSV data from the web into a dataframe is just like in R: `read_csv(data_url)` except that `read_csv()` is not built-in but in `pandas` (aliased to `pd`), so to find it, you need to run `pd.read_csv(data_url)`.

- This command fetches the data if they exist at the site. In Colab, the data frame is shown in a reduced form, as a smart table.

```
pd.read_csv(data_url)
```

- Include exception handling with `try` `except`: When interacting with the web, you always want to handle potential exceptions - e.g. data not available or corrupted, because of link or network errors.

```
try:
    df = pd.read_csv(data_url)
    print("Dataset downloaded and loaded successfully!")
except Exception as e:
    print(f"Error loading dataset: {e}")
    print("Please check your internet connection or the URL.")
```

```
Dataset downloaded and loaded successfully!
```

- The statement `print(f"Error loading dataset: {e}")` contains a so-called `f-string`, a formatted string that allows you to print variables inside strings.
- `Exception` is a class of several different exceptions, errors and warnings that can occur, see here for the full list.

Load the data into a file first

- You can also load the data into a file first; occasionally, this is something you want to do (especially for small datasets).

- In Python, you open a file socket (aka descriptor) and write to it. Since the data are on the web, you need to load `urllib`:

```
import urllib.request

data_url = 'https://tinyurl.com/iris-data-csv' # raw csv data
urllib.request.urlretrieve(data_url,"iris.csv") # retrieves data

import os
print(os.system('ls -l iris.csv')) # prints long file information &
                                  # exit status (0 = success)
print(os.system('head -n 2 iris.csv')) # prints top two lines of file
                                       # & exit status
```

```
-rw-rw-r-- 1 aletheia aletheia 3858 Sep  4 07:13 iris.csv
0
```

```
sepal_length,sepal_width,petal_length,petal_width,species
5.1,3.5,1.4,0.2,setosa
0
```

- If you want to see and handle errors explicitly, you need to separate the writing from the retrieving using *context management:

```
import urllib.request

with urllib.request.urlopen(data_url) as response: # context: open URL
    with open('data.csv', 'wb') as f: # write bytes to new local file
        f.write(response.read()) # read whole response, write it out

print(os.system('ls -l data.csv')) # prints long file information &
                                  # exit status (0 = success)
```

```
-rw-rw-r-- 1 aletheia aletheia 3858 Sep  4 07:13 data.csv
0
```

Explore the data

- This is where the fun begins: You now get to look at the data with the eye of a detective. We already saw what happened when we try to print the entire `DataFrame`:

```
print(df)
```

```
     sepal_length  sepal_width  petal_length  petal_width    species
0             5.1          3.5           1.4          0.2     setosa
1             4.9          3.0           1.4          0.2     setosa
2             4.7          3.2           1.3          0.2     setosa
3             4.6          3.1           1.5          0.2     setosa
4             5.0          3.6           1.4          0.2     setosa
..            ...          ...           ...          ...        ...
145           6.7          3.0           5.2          2.3  virginica
146           6.3          2.5           5.0          1.9  virginica
147           6.5          3.0           5.2          2.0  virginica
148           6.2          3.4           5.4          2.3  virginica
149           5.9          3.0           5.1          1.8  virginica

[150 rows x 5 columns]
```

- We are protected against overflow. You can also specify if you want to look at the top or bottom N rows only:

```
print(df.head(3))
print(df.tail(3))
```

```
   sepal_length  sepal_width  petal_length  petal_width species
0           5.1          3.5           1.4          0.2  setosa
1           4.9          3.0           1.4          0.2  setosa
2           4.7          3.2           1.3          0.2  setosa
     sepal_length  sepal_width  petal_length  petal_width    species
```

```
147           6.5          3.0           5.2          2.0  virginica
148           6.2          3.4           5.4          2.3  virginica
149           5.9          3.0           5.1          1.8  virginica
```

- Notice how the computer knows to apply `head` to `df`: The dot operator `.` binds the method `pandas.head()` to the `pandas` object `df`.

Optional: how `locals()` actually works `skip`

- Again, there is a safe way to do this that checks if the `DataFrame` is actually loaded and available as a local variable in `locals()`. This is Python's equivalent for `ls()`.

```
[print(l) for l in locals() if not l.startswith('_')]
```

```
tty
readline
ast
pd
f
sys
m
data_url
df
urllib
os
response
```

Confirm `df` is loaded

- The final command performs the check:

```
if 'df' in locals():
    print(df.head())
else:
    print("DataFrame 'df' not found. Run previous cell to load it?")
```

```
   sepal_length  sepal_width  petal_length  petal_width species
0           5.1          3.5           1.4          0.2  setosa
1           4.9          3.0           1.4          0.2  setosa
2           4.7          3.2           1.3          0.2  setosa
3           4.6          3.1           1.5          0.2  setosa
4           5.0          3.6           1.4          0.2  setosa
```

- The error message suggests a common issue in notebooks: When you come back to it after a while, your instance will have been disconnected, and your session has ended. You now must run all cells up to this point to repopulate the session, using the `Runtime -> Run all` tab at the top.

- Another useful function is `pandas.DataFrame.info` - that's the equivalent of `str` in R.

```
print(df.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   sepal_length  150 non-null    float64
 1   sepal_width   150 non-null    float64
 2   petal_length  150 non-null    float64
 3   petal_width   150 non-null    float64
 4   species       150 non-null    object
dtypes: float64(4), object(1)
memory usage: 6.0+ KB
None
```

- Here is a tutorial introduction to `pandas` data structures.
- Lastly, let's get a glimpse of the statistical properties of the data:

```
print(df.describe())
```

```
       sepal_length  sepal_width  petal_length  petal_width
count    150.000000   150.000000    150.000000   150.000000
mean       5.843333     3.057333      3.758000     1.199333
std        0.828066     0.435866      1.765298     0.762238
min        4.300000     2.000000      1.000000     0.100000
25%        5.100000     2.800000      1.600000     0.300000
50%        5.800000     3.000000      4.350000     1.300000
75%        6.400000     3.300000      5.100000     1.800000
max        7.900000     4.400000      6.900000     2.500000
```

- To visualize these, one would use a boxplot.

Plotting with Python

- Making plots, or graphs, is another way of exploring the data. Two popular graphics packages in Python are `matplotlib` and `seaborn`.
- `matplotlib` is a low-level plotting library that can draw figures, axes, labels, lines etc.
- We're going to `import` a sub-library of `matplotlib`, `pyplot`, with the alias `plt`.
- To **practice** what you learnt earlier, check that `plt` is indeed loaded into the Python session namespace.

```
import matplotlib.pyplot as plt
print('matplotlib' in sys.modules)
```

```
True
```

- A common source of error: The alias `plt` is not loaded but it's the name that you need to use to access methods. Aliases like `plt` are local variable bindings in your namespace - they do not appear in `sys.modules`. When you do `import matplotlib.pyplot as plt`, Python:
    1. Loads the module `matplotlib.pyplot` if not already loaded.
    2. Stores it in `sys.modules` under its true name.
    3. Binds a reference to that module object to the name `plt` in your current namespace

- What also might have happened is that "no module named matplotlib" is found because the library was not installed. To install, use `pip` on the command line:

```
!pip install matplotlib
```

Decide what you want to plot

- So far we haven't cared at all what these data are and what they mean. If you don't want to do a web search, this is a perfect entry for AI.
- Open the Gemini console, and ask:

```
What is in the `Iris` dataset, and where does it come from?
```

- Once you got the answer, you can ask Gemini to add it to the notebook. If it does not give you a link to look up more, ask for it. If it gives you a Wikipedia link, ask for another. The best source is the UCI archive (see below):

> "The `Iris` dataset is a classic and widely used dataset in machine learning and statistics. It contains 150 samples of iris flowers, with each sample having four features: sepal length, sepal width, petal length, and petal width, all measured in centimeters. Each sample is also labeled with one of three species of iris: Setosa, Versicolor, or Virginica. It's often used for classification tasks to predict the species of an iris flower based on its measurements. The dataset was introduced by the British statistician and biologist Ronald Fisher in 1936.
>
> For a comprehensive reference, you can find the Iris dataset at the UCI Machine Learning Repository."

- Now you know! Now the columns of the `DataFrame` make sense. Print the `info()` again:

```
print(df.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
 #   Column        Non-Null Count  Dtype  
---  ------        --------------  -----  
 0   sepal_length  150 non-null    float64
 1   sepal_width   150 non-null    float64
 2   petal_length  150 non-null    float64
 3   petal_width   150 non-null    float64
 4   species       150 non-null    object 
dtypes: float64(4), object(1)
memory usage: 6.0+ KB
None
```

- What additional information does this table give us?

1. There are for numeric (0-3) and one categorical feature (`species`).
2. The numeric features are represented as floating-point numbers.
3. There are no missing values (`NaN`).

- Without this semantic detour, plotting makes no sense because you have to first identify what you want to plot and why, before you decide how. And the plotting functions require decisions on coordinates and other customizations from you.

- What can we plot here?

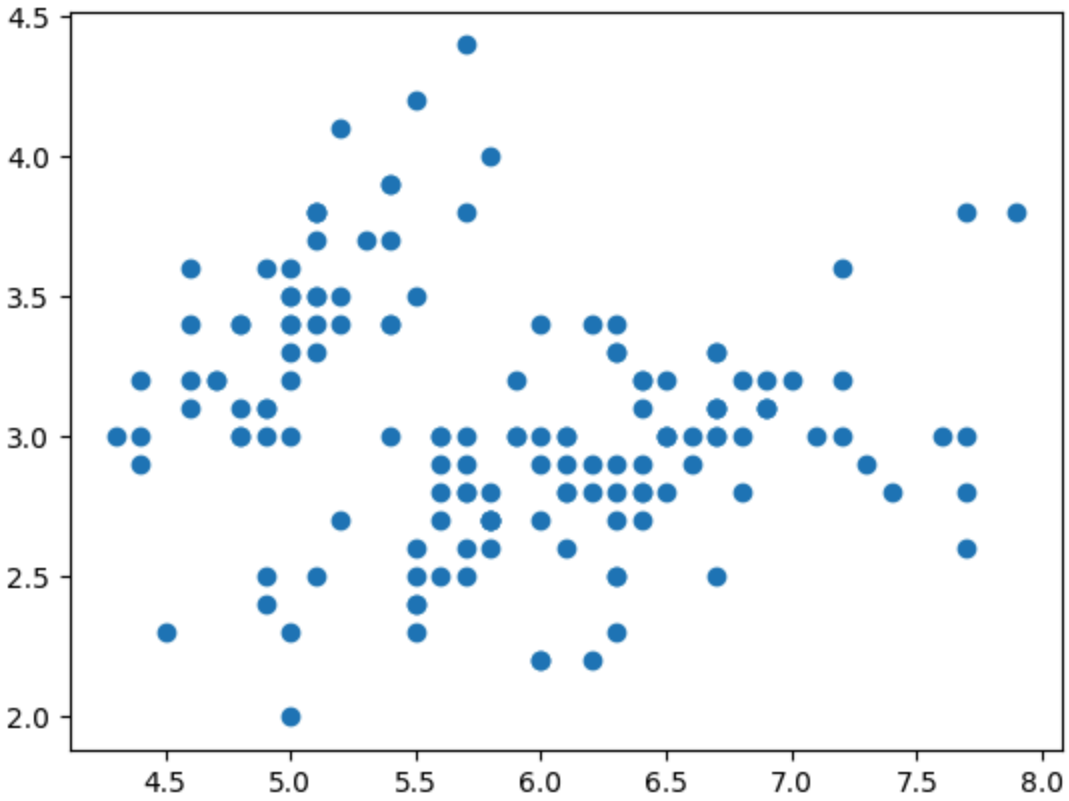
    1. We can create **scatterplots** of the numeric variables (in 2D or 3D) for all `species` to see how they vary by `species`.
    2. We can create **barplots** of the mean value of each feature per `species` to compare average measurements across groups.
    3. We can create **boxplots** for each feature against the `species` to show the statistical properties of the distributions.

- In this notebook, we will only demonstrate scatterplots. For a collection of hundreds of charts made with Python using different graphics packages, see [The Python Graph Gallery](#).

Create a simple plot with `matplotlib`

- First, we create a simple plot that shows the relationship of two features for all `species`, `sepal_length` and `sepal_width`.

```
plt.scatter(x=df['sepal_length'],
            y=df['sepal_width'])
plt.savefig("../img/iris.png")
```

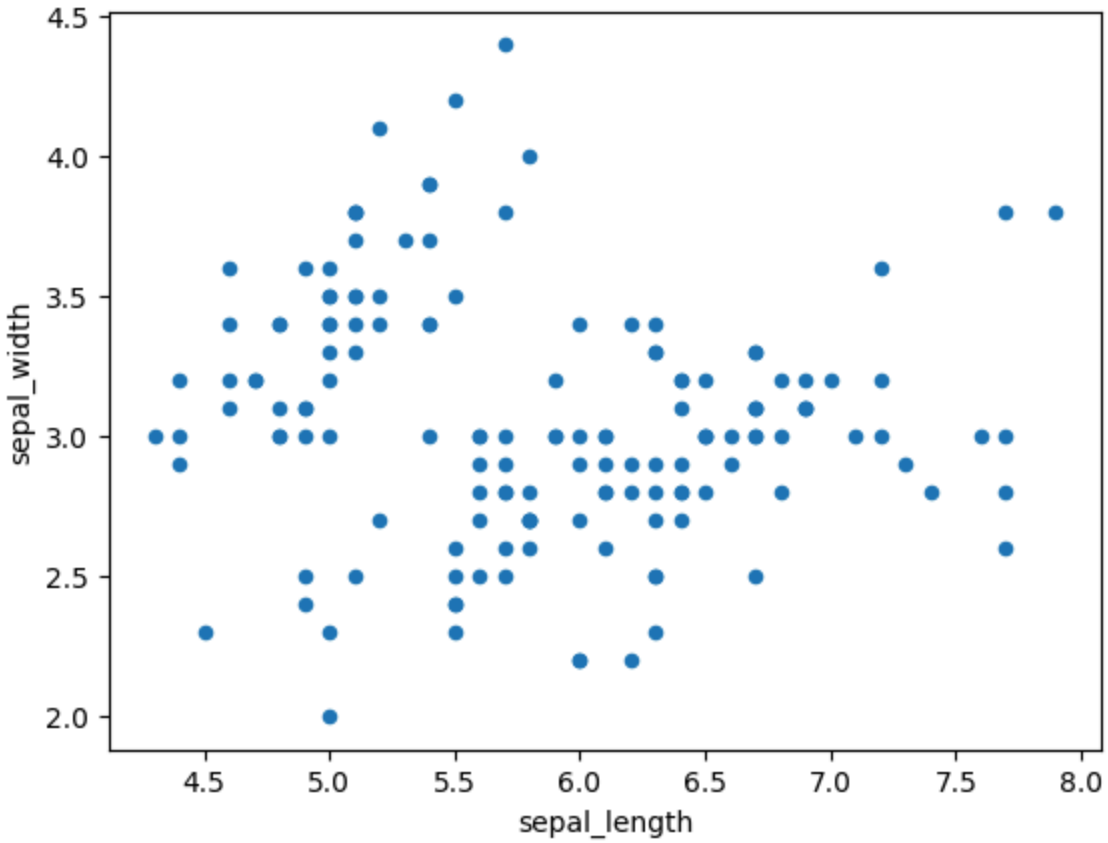


- To access the features, which are stored as vectors in the columns of the `DataFrame`, we used the subscript or index operator `[]` and the names of the features. The plotting function then picks the values from the column one after the next in order of appearance/sampling.

Optional: customization, grouping by hand, and debugging `skip`

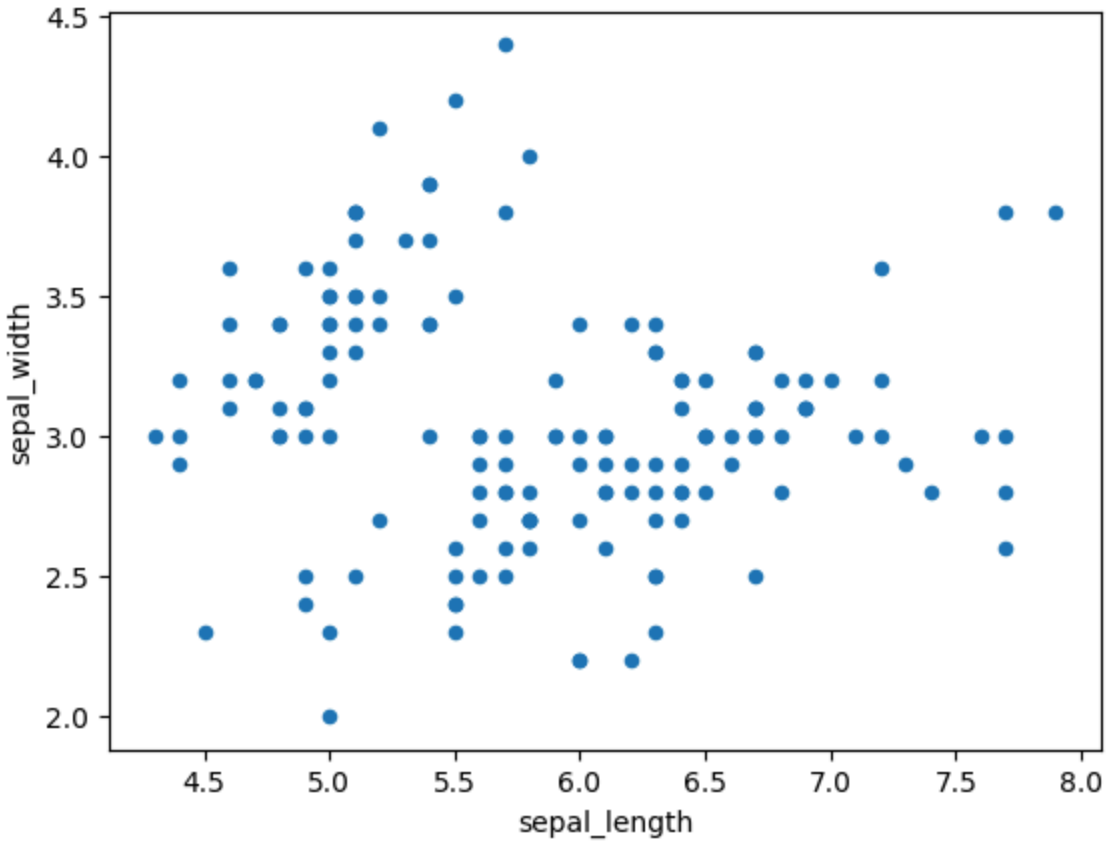
- This plot is easier to understand with some minimal customization, like a `title()`, and labels for x and y coordinates. Unlike other commands, we cannot just add these to an existing plot but we must clean the graphics and create a new plot.

```
plt.clf() # cleans the slate
plt.scatter(df['sepal_length'],df['sepal_width'])
plt.title("Sepal Width vs. Sepal Length (Iris Dataset)")
plt.xlabel("Sepal Length (cm)")
plt.ylabel("Sepal Width (cm)")
plt.savefig("../img/iris2.png")
```



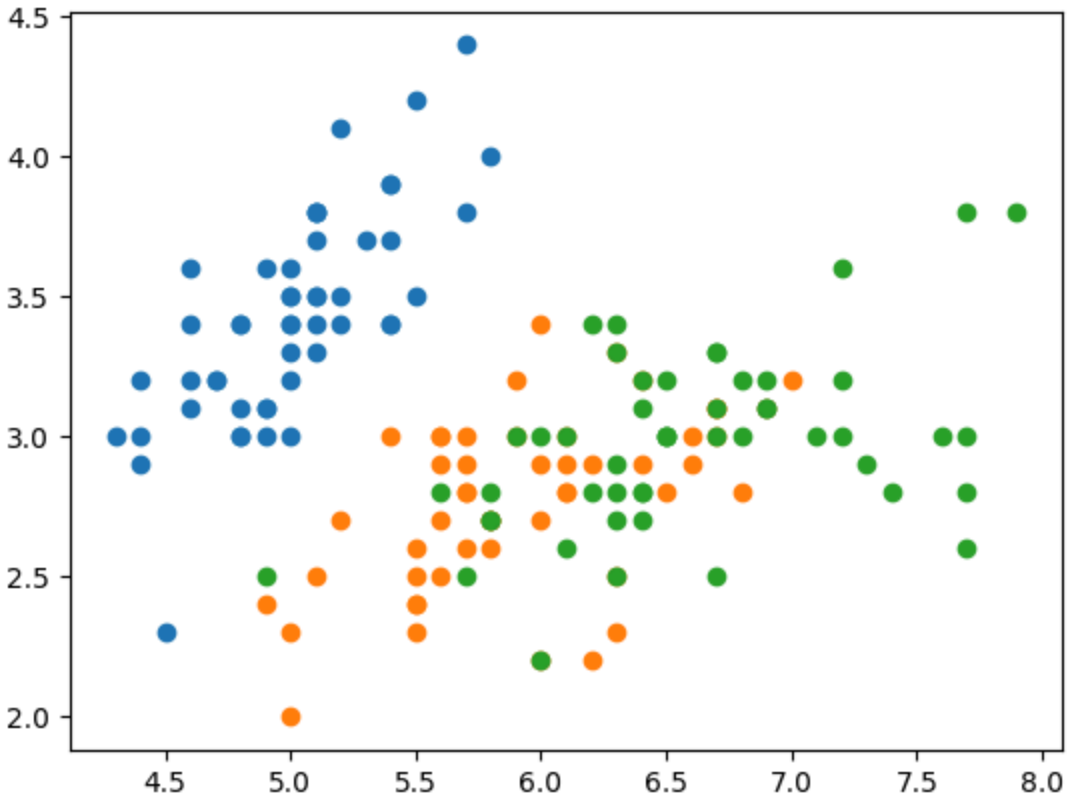
- There is no grouping here, no colors by species, just a straight plot of all data in the length and width columns. In this way, we are losing information that's actually there.
- Since Python functions can be "chained" (the equivalent of R's pipelining), you can write this much shorter with a `pandas` wrapper:

```
df.plot.scatter(x='sepal_length',y='sepal_width')
plt.savefig("../img/iris2.png")
```



- The `Iris` dataset is often used to demonstrate the abilities of machine learning models to cluster the three species. To show them, we need to subset the data (group them) by species and add the species information during the plotting. Python will then automatically assign different colors:

```
for s in df['species'].unique():  # iterate over the 3 unique species names
    sub = df[df['species']==s]    # filter df to rows where species matches s
    plt.scatter(x=sub['sepal_length'],
                y=sub['sepal_width'],
                label=s)
```



- Let's see how this works in detail:

  1. `df['species'].unique()` returns a list of the three unique species names:

     ```
     print(df['species'].unique())
     ```

     ```
     ['setosa' 'versicolor' 'virginica']
     ```

  2. `df['species']==s` produces a series of Booleans: `True` for the current value of `species`, `False` otherwise.

     ```
     s = 'setosa'
     print(df['species']==s)
     ```

     ```
     0       True
     1       True
     2       True
     3       True
     4       True
             ...
     145    False
     146    False
     ```

```
147    False
148    False
149    False
Name: species, Length: 150, dtype: bool
```

3. `df[...]` uses the Boolean series (or logical flag vector) as a filter and keeps only rows where the value is `True`: a total of 50 rows x 5 columns for each `species`:

```
sub = df[df['species']==s]
print(sub.shape)
```

```
(50, 5)
```

4. `plt.scatter(x=sub[...], y=sub[...])` then plots each group. The `label` parameter assigns a different color to each group automatically.

- The grouping by masking row values using a Boolean index vector (or Series in Python) is worth showing again because it's a common trick:

```
s = 'setosa'
mask = df['species']==s  # Step 1: Boolean series (flag vector)
sub = df[mask]           # Step 2: Filter rows using mask
print(mask.head())       # Show True/False values
print(sub.head())        # Show the filtered rows
```

```
0    True
1    True
2    True
3    True
4    True
Name: species, dtype: bool
   sepal_length  sepal_width  petal_length  petal_width species
0           5.1          3.5           1.4          0.2  setosa
1           4.9          3.0           1.4          0.2  setosa
2           4.7          3.2           1.3          0.2  setosa
3           4.6          3.1           1.5          0.2  setosa
4           5.0          3.6           1.4          0.2  setosa
```

- When I first wrote this last code, I made an instructive mistake. This command does not work but aborts with a `KeyError`. What's wrong?

```
for s in df['species'].unique():
    sub = df[df['species'==s]]
    plt.scatter(x=sub['sepal_length'],
                y=sub['sepal_width'],
                label=s)
```

- Answer:

  `df[df[X]]` looks for a key to a column `X`. But there is no column name `'species'==s` - hence the `KeyError`. You need to correct this to `df['species']==s` to get a key/column name to be compared with `s`.

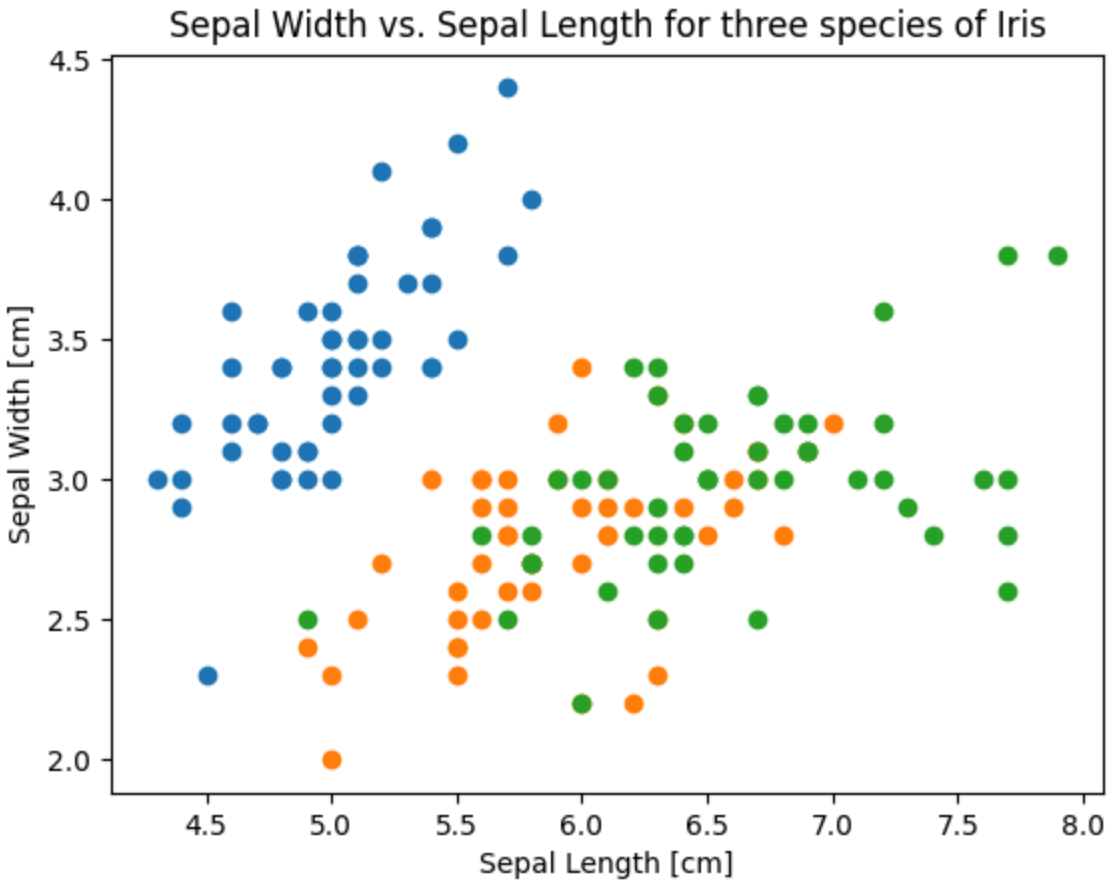
- Subsetting rule to remember:

  `df['col']` -> select a column `df['col'] === value` -> Boolean Series (aka vector)
  `df[Boolean_series]` -> row filter (when Boolean is `True`)

- For more information on `matplotlib`, see Géron's tutorial.

- As a small challenge, copy the previous plot and add a title and axis labels as before.

```
plt.clf()
plt.title("Sepal Width vs. Sepal Length for three species of Iris")
plt.xlabel("Sepal Length [cm]")
plt.ylabel("Sepal Width [cm]")
for s in df['species'].unique():
    sub = df[df['species']==s]
    plt.scatter(x=sub['sepal_length'],
                y=sub['sepal_width'],
                label=s)
plt.savefig("../img/iris4")
```



- Were you trying to put the title and label definitions inside the loop right after the plt.scatter() call? This would work, too, but it would print them on top of one another three times.
- In Python, the preparation of the figure and the plotting of the data, are interleaved. You will see that even more clearly with the next graphics package, seaborn.

Create a fancy plot with `seaborn`

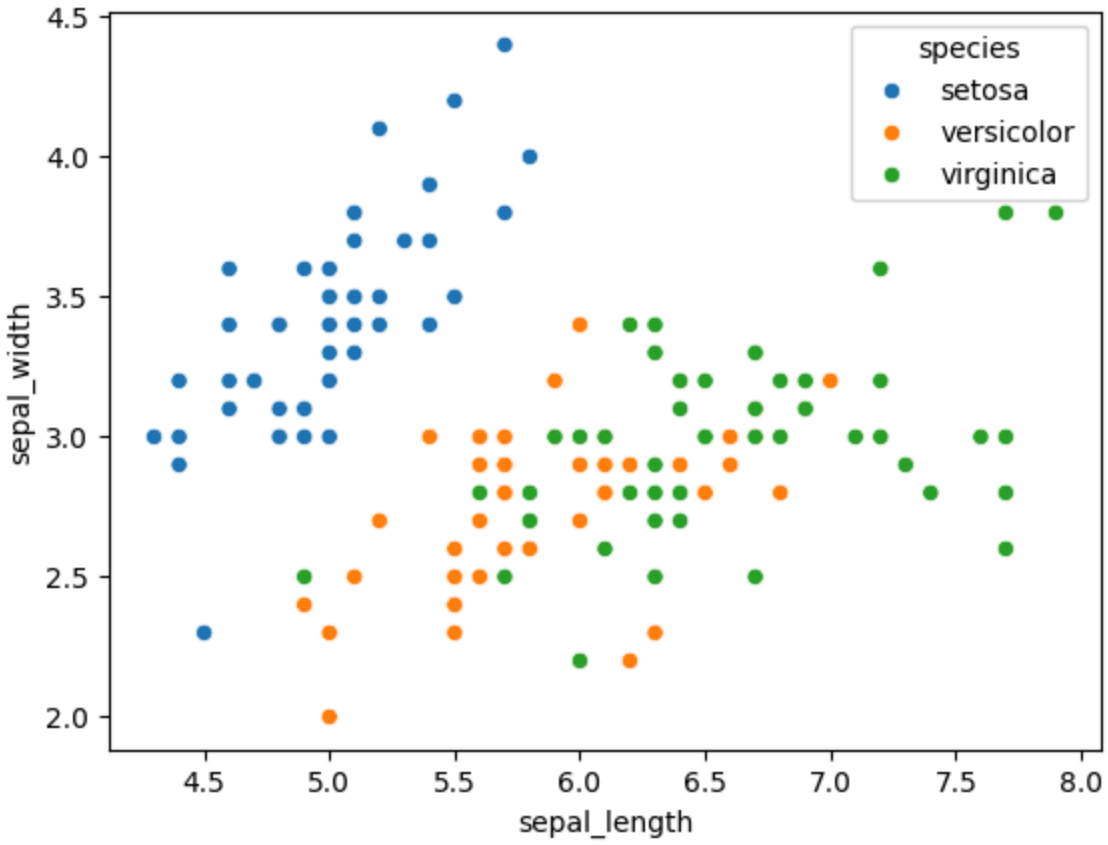
- The `seaborn` library is built on top of `matplotlib` and allows automatic grouping of the data by the variable `species`.
- First, we load the library, and check if the loading worked by asking if the alias `sns` is available in the current namespace:

```
import seaborn as sns
print('sns' in locals())
```

```
True
```

- Now we create a fancy plot with grouping by `species` in one line of code:

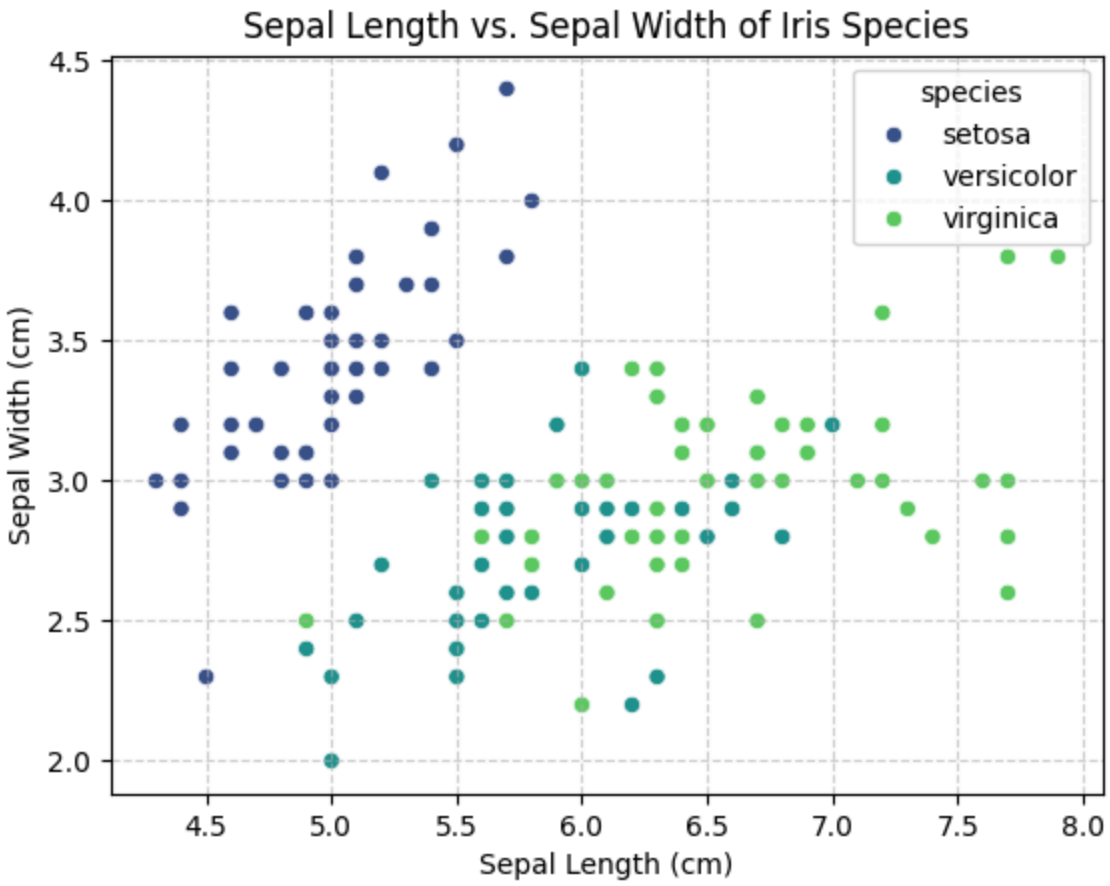
```
sns.scatterplot(data=df,
                x='sepal_length',
                y='sepal_width',
                hue='species')
```



Optional: seaborn customization (title, labels, grid) `skip`

- The customization is done using `matplotlib` functions. We copy the previous code and add title, labels, and, for greater readability, a grid:

```
# base plot
sns.scatterplot(data=df,
                x='sepal_length',
                y='sepal_width',
                hue='species',
                palette='viridis')
## customization
plt.title('Sepal Length vs. Sepal Width of Iris Species')
plt.xlabel('Sepal Length (cm)')
plt.ylabel('Sepal Width (cm)')
plt.grid(True, # show a grid
         linestyle='--', # gridline style
         alpha=0.6) # gridline transparency
plt.savefig("../img/iris6.png")
```



Summary

	Command / Concept	Definition
1	`import pandas as pd`	Load pandas library as alias pd
2	`sys.modules`	Dict of all currently loaded modules
3	`any("pandas" in m for m in ...)`	Test whether pandas is loaded
4	`dir(pd)`	List names defined by pandas
5	`pd.read_csv(url)`	Read CSV data from URL into DataFrame
6	`try / except`	Handle runtime errors safely
7	`Exception`	Base class for runtime errors
8	`f"string {var}"`	Formatted string (f-string)
9	`print(df)`	Display entire DataFrame `df`
10	`df.head()`	Show first rows of DataFrame
11	`locals()`	Dict of current local variables
12	`'df' in locals()`	Check if DataFrame exists
13	`import matplotlib.pyplot as plt`	Load matplotlib plotting interface
14	`plt.scatter(x,y)`	Draw scatter plot
15	`plt.xlabel(), plt.ylabel()`	Label plot axes
16	`plt.savefig(file)`	Save plot to file
17	`plt.show()`	Save plot to screen display
18	`df.plot.scatter(...)`	pandas wraps matplotlib scatter
19	`df['col']`	Select column (Series)
20	`df['col'] =` value=	Boolean mask for rows
21	`df[mask]`	Filter rows using Boolean mask
22	`plt.figure(figsize=(w,h))`	Create new figure of given size
23	`figsize`	Physical plot size, not data scaling
24	`plt.grid(alpha=x)`	Add grid with transparency
25	`alpha`	Transparency (0=clear, 1=opaque)
26	`plt.legend()`	Show legend for labeled plot elements
27	`import seaborn as sns`	Load seaborn plotting library
28	`sns.scatterplot(...)`	Scatter plot with automatic grouping
29	`hue='var'`	Group data by categorical variable
30	`locals()`	Dictionary of current namespace entries

----------+

Programming Assignment (Home)

Test your understanding of "Getting started with Colab (and Python)" by creating a notebook for the Palmer Penguin dataset available for you online from: tinyurl.com/palmer-data-csv.

Tasks:

1. Create a new Colab notebook.
2. Load `pandas` with the alias `pd`.
3. Load the dataset into a `DataFrame` df with `pd.read_csv`
4. Check if the dataset is loaded in the current namespace by checking if `pd` is in `locals()`.
5. Display the first few lines of the dataset with `pd.head()`
6. Make a simple plot of the `bill_length_mm` as a function of `bill_depth_mm` features using `matplotlib.pyplot` as `plt`.
7. Make a fully customized plot of the dataset grouping by `species` using `seaborn` as `sns`.
8. Make sure that the notebook is shared.
9. Upload the (shareable) URL to your finished notebook to Teams.
10. Use markdown for headlines and comments (outside of the code blocks), and created a structured notebook, not just a bunch of code.
11. Ask Gemini for a critique of your solution and add its critique at the end of the notebook.

Let me know if you have any problems!

Footnotes:

[1] IPython is an enhanced interactive Python shell that replaces the standard Python REPL with features designed for scientific computing - including integration with Jupyter Notebooks (like Colab).

Author: Marcus Birkenkrahe
Created: 2026-09-04 Fri 07:13